Assembly for Reverse Engineering

Memory

Barak Gonen

Subjects

- Accessing memory
- Hands on- using the .data section
- Little – Big endian
- Pointers
- Special challenge – write code like malware does

Memory

- Each BYTE has a memory location
  - Byte- 8 bits
  - Word – 2 bytes
  - Double word – 4 bytes
- In 32 bit architecture, each memory reference may refer to a byte, a word or a double word

Defining Special Variables

- Defining an array:
  - DUP means ‘duplicate’

```
my_array db 5 dup(0x01)
```

- Defining a string:
  - 13 means ‘\r’
  - 10 means ‘\n’

```
my_str  db  'Hello class!',13,10
```

Defining Variables in Memory

- db – define byte
- dw – define word
- dd – define double
- Value can be initialized:
- …Or not initialized:

```
my_var db 7
my_other_var db ?
```

Defining Variables – cont.

- What will be the memory map if the following variables were defined?
- Try it!

```
section '.data' data readable writeable
    a   dd  3
        db  0x65, 0x43, 0x21
    b   dw  4 dup(?)
    c   dd  3
        db  5 dup(0xFE)
```

Accessing memory

- The address is highly required ☺
- We use [ ] to indicate the value which is stored at the given address
- Direct memory address:
  - [0x00401000]
  - [var1] – when var1 is defined in data segment
- Indirect addressing:
  - [ebx] – if ebx is storing the address
  - [ebx + const] – fixed offset
  - [ebx + esi] – great for looping on arrays

Exercises – Memory

1. Define x to hold the value “8” in a byte size
2. Copy the value of x to AL
3. Copy the value at address 0x00401000 to AH
4. Copy the address of x to EBX
5. Define y to hold the value “9” in a byte size
6. Copy the value of [ebx] to CL
7. Copy the value of [ebx+1] to CH
8. Attempt to copy the value of x to AX
9. Attempt to copy the value at 0x00402000 to DL

Exercise – Kill Beale

Beale.asm (google Beale cipher for an interesting historical story) includes part of a code which deciphers an encrypted string.

Add the required code so the code will conduct its purpose

Little Endian

- How ‘number’ is stored in memory?

```
section '.data' data readable writeable
    number  dd   0x6789ABCD
```

- Little Endian: Significant bytes stored in higher memory address

Address	Hex dump							
00401000	CD	AB	89	67	00	00	00	00
00401008	00	00	00	00	00	00	00	00

LEA

Load Effective Address

Actually enables several calculations in one instruction

Calculates equations of the form " register*a + register + b ", where a = 1, 2, 4, or 8

```
mov     ebx, 5
mov     ecx, 1
lea     eax, [ebx*8+ecx+5]
call    print_eax
```

Label

- Provide a “name” to a memory address containing code
- During assembling process, name will be replaced by actual address
- Think: why is it useful?
- Let’s try it. Assemble and follow in disassembler:

```
    xor     eax, eax
here:
    mov     ebx, here
    mov     eax, ebx
    call    print_eax
```

Exercise – Possible

- https://data.cyber.org.il/reversing/possible.pdf

Pointers

```
section '.data' data readable writeable
    x    db 0x8
    ptrx     dd ?

section '.text' code readable executable
; ====================================

start:
    xor     eax, eax
    mov     al, [x]
    call    print_eax
    mov     eax, x
    call    print_eax
    mov     [ptrx], eax
    mov     eax, ptrx
    call    print_eax
    mov     eax, [ptrx]
    call    print_eax
    mov     ebx, [ptrx]
    xor     eax, eax
    mov     al, byte [ebx]
    call    print_eax
```

- Example of PCHAR
- Copy and follow in IDA
- https://data.cyber.org.il/reversing/pointer.asm

Another Look At Pointers

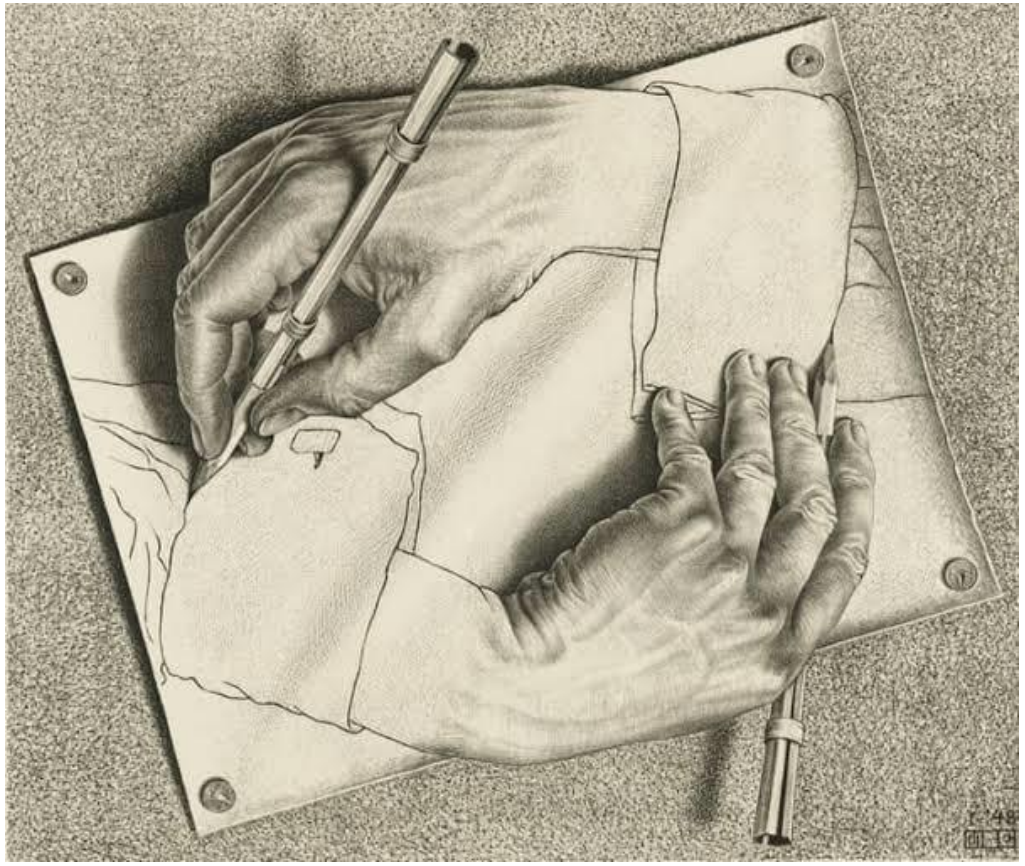
```
int x, z;
int *y;

int main()
{
    x = 5;
    y = &x;
    z = *y;
    printf("%d %d %d\n", y, *y, z);
    return 0;
}
```

- Use Visual Studio
  - Breakpoint
  - Disassembly window
- What is the assembly code for “z = *y” ?

ClassWork

- Little-Big
- Master Escher



QUESTIONS